

Loan Eligibility Case Study using Pandas

This case study automates the loan eligibility process based on customer details provided during an online application. The aim is to analyze and clean the dataset, derive meaningful insights, and prepare the data for further machine learning modeling.

We will perform:

- Data loading and inspection
 - Missing data handling
 - Data transformation
 - Visualization
- using Pandas, NumPy, Matplotlib, and Seaborn.

1. Loading Data in Pandas DataFrame

Explanation: First, we import necessary libraries and load the loan dataset into a DataFrame.

```
1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  df = pd.read_csv(r"D:\Case Study\Python Case Study\LoanData (1).csv")
7
8  print("File loaded successfully!")
```

```
PS D:\Microsoft VS Code\code\Code> & C:/Users/sathu/AppData/Local/Programs/Python/Python313/python.exe "d:/Microsoft VS Code/code/Code/py
thon_casestudy.py"
File loaded successfully!
```

2. Printing First Few Rows of Data

Explanation: To get an idea of the dataset, we can print the top and bottom rows.

```
# First 5 rows
print(df.head())

# Last 5 rows
print(df.tail())
```

```
File loaded successfully!
  Loan_ID Gender Married Dependents Education ... LoanAmount Loan_Amount_Term Credit_History Property_Area Loan_Status
0 LP001002 Male    No         0 Graduate ...      NaN          360.0           1.0         Urban         Y
1 LP001003 Male   Yes         1 Graduate ...    128.0          360.0           1.0         Rural         N
2 LP001005 Male   Yes         0 Graduate ...     66.0          360.0           1.0         Urban         Y
3 LP001006 Male   Yes         0 Not Graduate ...    120.0          360.0           1.0         Urban         Y
4 LP001008 Male   No         0 Graduate ...    141.0          360.0           1.0         Urban         Y

[5 rows x 13 columns]
  Loan_ID Gender Married Dependents Education ... LoanAmount Loan_Amount_Term Credit_History Property_Area Loan_Status
609 LP002978 Female    No         0 Graduate ...     71.0          360.0           1.0         Rural         Y
610 LP002979 Male   Yes        3+ Graduate ...     40.0          180.0           1.0         Rural         Y
611 LP002983 Male   Yes         1 Graduate ...    253.0          360.0           1.0         Urban         Y
612 LP002984 Male   Yes         2 Graduate ...    187.0          360.0           1.0         Urban         Y
613 LP002990 Female    No         0 Graduate ...    133.0          360.0           0.0       Semiurban         N

[5 rows x 13 columns]
```

3. Printing the column names of the DataFrame

```
# Printing column names of the DataFrame
print("Column Names:")
print(df.columns.tolist())
```

```
Column Names:
['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status']
```

4. Summary of the DataFrame

```
# Summary of the DataFrame
print("Summary of the DataFrame:")
df.info()
```

```
Summary of the DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Loan_ID               614 non-null   object 
 1   Gender                601 non-null   object 
 2   Married               611 non-null   object 
 3   Dependents            599 non-null   object 
 4   Education             614 non-null   object 
 5   Self_Employed         582 non-null   object 
 6   ApplicantIncome       614 non-null   int64  
 7   CoapplicantIncome     614 non-null   float64 
 8   LoanAmount            592 non-null   float64 
 9   Loan_Amount_Term      600 non-null   float64 
10  Credit_History        564 non-null   float64 
11  Property_Area         614 non-null   object 
12  Loan_Status           614 non-null   object 
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

5. Descriptive Statistical Measures of the DataFrame

```
# Descriptive statistical measures of the DataFrame
print("Descriptive Statistics:")
print(df.describe())
```

```
Descriptive Statistics:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

6. Handling Missing Data

```
# Checking for missing values
print("Missing values in each column:")
print(df.isnull().sum())

# Filling missing values (categorical with mode, numerical with median)
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mode()[0])
df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].median())
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].median())

print("\nMissing values filled successfully.")
```

```
Missing values in each column:
Loan_ID          0
Gender           13
Married          3
Dependents       15
Education        0
Dependents       15
Education        0
Education        0
Self_Employed    32
Self_Employed    32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term 14
Credit_History  50
Property_Area    0
Loan_Status      0
dtype: int64

Missing values filled successfully.
```

7: Sorting DataFrame values:

```
# Sort the data by 'ApplicantIncome' in descending order
sorted_df = df.sort_values(by='ApplicantIncome', ascending=False)

# Display top 5 records after sorting
print("Top 5 applicants with highest income:")
print(sorted_df[['Loan_ID', 'ApplicantIncome', 'LoanAmount']].head())
```

Top 5 applicants with highest income:

	Loan_ID	ApplicantIncome	LoanAmount
409	LP002317	81000	360.0
333	LP002101	63337	490.0
171	LP001585	51763	700.0
155	LP001536	39999	600.0
185	LP001640	39147	120.0

8: Merge DataFrames

```
# Creating two small DataFrames to demonstrate merging
df1 = df[['Loan_ID', 'ApplicantIncome', 'LoanAmount']].head(5)
df2 = df[['Loan_ID', 'Credit_History', 'Loan_Status']].head(5)

# Merging on 'Loan_ID'
merged_df = pd.merge(df1, df2, on='Loan_ID')

print("Merged DataFrame:")
print(merged_df)
```

Merged DataFrame:

	Loan_ID	ApplicantIncome	LoanAmount	Credit_History	Loan_Status
0	LP001002	5849	128.0	1.0	Y
1	LP001003	4583	128.0	1.0	N
2	LP001005	3000	66.0	1.0	Y
3	LP001006	2583	120.0	1.0	Y
4	LP001008	6000	141.0	1.0	Y

9: Apply Function

```
# Creating a new column to categorize income level
def categorize_income(income):
    if income < 2500:
        return 'Low'
    elif income < 6000:
        return 'Medium'
    else:
        return 'High'

df['Income_Level'] = df['ApplicantIncome'].apply(categorize_income)

print("Income levels assigned using apply():")
print(df[['ApplicantIncome', 'Income_Level']].head())
```

```
Income levels assigned using apply():
   ApplicantIncome  Income_Level
0             5849         Medium
1             4583         Medium
2             3000         Medium
3             2583         Medium
4             6000          High
```

10: By using the lambda operator

```
# Calculate average loan amount
avg_loan = df['LoanAmount'].mean()

# Create a new column that flags loans above the average
df['Above_Avg_Loan'] = df['LoanAmount'].apply(lambda x: 'Yes' if x > avg_loan else 'No')

print(f"Average Loan Amount: {avg_loan:.2f}")
print(df[['LoanAmount', 'Above_Avg_Loan']].head())
```

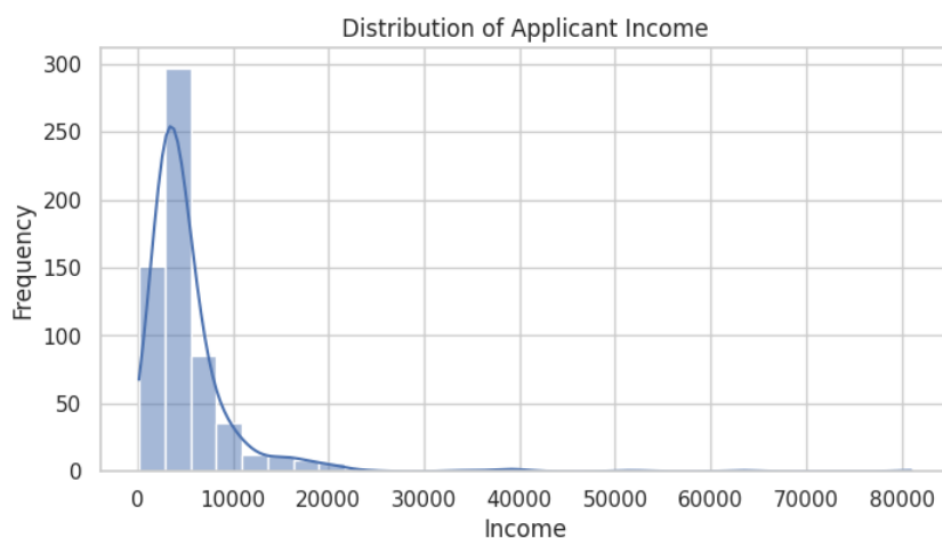
```
Average Loan Amount: 145.75
   LoanAmount Above_Avg_Loan
0         128.0             No
1         128.0             No
2          66.0             No
3         120.0             No
4         141.0             No
```

11: Visualizing DataFrame

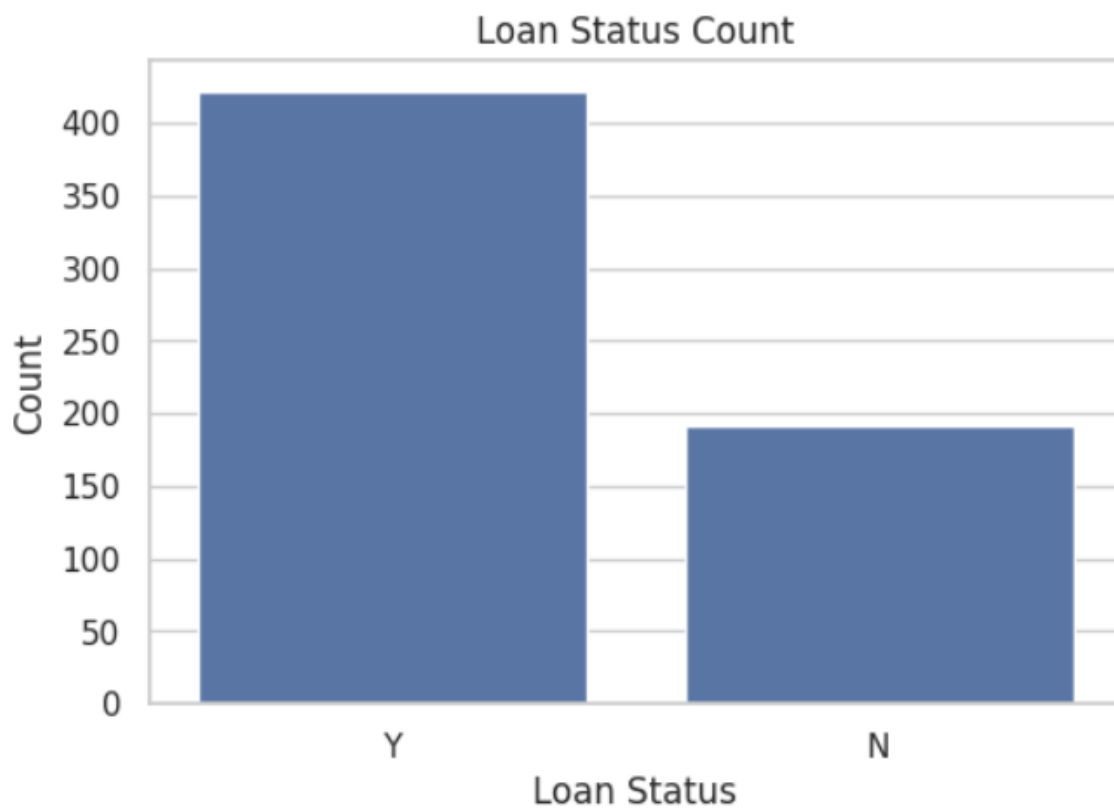
```
import matplotlib.pyplot as plt
import seaborn as sns

# Set style
sns.set(style='whitegrid')

# Plot 1: Distribution of Applicant Income
plt.figure(figsize=(8, 4))
sns.histplot(df['ApplicantIncome'], kde=True, bins=30)
plt.title('Distribution of Applicant Income')
plt.xlabel('Income')
plt.ylabel('Frequency')
plt.show()
```




```
# Plot 2: Countplot of Loan Status
plt.figure(figsize=(6, 4))
sns.countplot(x='Loan_Status', data=df)
plt.title('Loan Status Count')
plt.xlabel('Loan Status')
plt.ylabel('Count')
plt.show()
```



```
# Plot 3: Boxplot of LoanAmount by Education
plt.figure(figsize=(8, 5))
sns.boxplot(x='Education', y='LoanAmount', data=df)
plt.title('Loan Amount by Education Level')
plt.xlabel('Education')
plt.ylabel('Loan Amount')
plt.show()
```

